Foundation Models for Partial Differential Equations

Abdullah Ahmed

University of Maryland

November 17, 2025

Outline

Why use Deep Learning for PDEs?

Why use Deep Learning for PDEs?

- PDEs model physical phenomena everywhere (fluid dynamics, climate, materials, electromagnetism).

Why use Deep Learning for PDEs?

- PDEs model physical phenomena everywhere (fluid dynamics, climate, materials, electromagnetism).
- Classical numerical solvers like FEM are accurate but can be computationally expensive for repeated queries or in higher dimensions.

Why use Deep Learning for PDEs?

- PDEs model physical phenomena everywhere (fluid dynamics, climate, materials, electromagnetism).
- Classical numerical solvers like FEM are accurate but can be computationally expensive for repeated queries or in higher dimensions.
- Learned surrogates / operators can provide orders of magnitude speedups at inference time and enable embedding into bigger data-driven pipelines.

Why use Deep Learning for PDEs?

- PDEs model physical phenomena everywhere (fluid dynamics, climate, materials, electromagnetism).
- Classical numerical solvers like FEM are accurate but can be computationally expensive for repeated queries or in higher dimensions.
- Learned surrogates / operators can provide orders of magnitude speedups at inference time and enable embedding into bigger data-driven pipelines.
- Inverse problems of PDEs have extensive applications in fields involving sensing and reconstruction such as medical imaging and geological sensing.

Classical DL approaches for PDEs

Classical DL approaches for PDEs

- **PINNs** (Physics-Informed Neural Networks)(Rasissi et al. 2019): incorporate the PDE and boundary/initial conditions into the loss function; excel for inverse and data-scarce problems but can be difficult to scale or tune. Requires retraining for every distinct problem.

Classical DL approaches for PDEs

- **PINNs** (Physics-Informed Neural Networks)(Rasissi et al. 2019): incorporate the PDE and boundary/initial conditions into the loss function; excel for inverse and data-scarce problems but can be difficult to scale or tune. Requires retraining for every distinct problem.
- **DeepONet**(Lu et al. 2019): learns operators (mapping source/velocity/initial condition function $\rightarrow$ function) using a branch+trunk architecture. Has universal approximation property and does not require retraining for slightly different problem, but it is constrained by a fixed discretizations for queries.

Classical DL approaches for PDEs

- **PINNs** (Physics-Informed Neural Networks)(Rasissi et al. 2019): incorporate the PDE and boundary/initial conditions into the loss function; excel for inverse and data-scarce problems but can be difficult to scale or tune. Requires retraining for every distinct problem.
- **DeepONet**(Lu et al. 2019): learns operators (mapping source/velocity/initial condition function $\rightarrow$ function) using a branch+trunk architecture. Has universal approximation property and does not require retraining for slightly different problem, but it is constrained by a fixed discretizations for queries.
- **Neural Operators**(Kovachki et al. 2022): also learn maps between function spaces, but architecture ensures that it is discretization-invariant (can be queried at any point after training). Usually includes integral / spectral parameterizations, and is a universal framework that includes the state-of-the-art **FNO**, **GNO**, **MGNO**, **LNO**. Also has UAP.

Transformer-Based Neural Operators

Transformer-Based Neural Operators

- The attention mechanism is a special case of the integral kernel layer in the general neural operator.

Transformer-Based Neural Operators

- The attention mechanism is a special case of the integral kernel layer in the general neural operator.
- Most recent NOs utilize transformer-based architectures, using self-attention to characterize non-local interactions between sample points.

Transformer-Based Neural Operators

- The attention mechanism is a special case of the integral kernel layer in the general neural operator.
- Most recent NOs utilize transformer-based architectures, using self-attention to characterize non-local interactions between sample points.
- Some approaches use cross-attention type mechanisms to learn computationally-favorable latent spaces to work in (as opposed to the FNO, for example, where the latent space is fixed as the frequency domain).

Why we need "foundation models" for PDEs

Why we need "foundation models" for PDEs

- **Task specificity:** many operator models are trained per-PDE or per-domain and struggle to transfer across PDE families.

Why we need "foundation models" for PDEs

- **Task specificity:** many operator models are trained per-PDE or per-domain and struggle to transfer across PDE families.
- **Data cost:** high-fidelity PDE data are expensive to generate; pretraining can help but requires robust strategies.

Why we need "foundation models" for PDEs

- **Task specificity:** many operator models are trained per-PDE or per-domain and struggle to transfer across PDE families.
- **Data cost:** high-fidelity PDE data are expensive to generate; pretraining can help but requires robust strategies.
- **Geometry / BC diversity:** many architectures assume structured grids or periodic BCs, limiting applicability to irregular geometries.

Why we need "foundation models" for PDEs

- **Task specificity:** many operator models are trained per-PDE or per-domain and struggle to transfer across PDE families.
- **Data cost:** high-fidelity PDE data are expensive to generate; pretraining can help but requires robust strategies.
- **Geometry / BC diversity:** many architectures assume structured grids or periodic BCs, limiting applicability to irregular geometries.
- **Scaling:** large models (and large datasets) are needed to reach broad generalization — parallels with LLMs motivate foundation-model research for PDEs.

Definition

Definition

A PDE foundation model (FM) aims to be:

- Pretrained on a *diverse* collection of PDE datasets (different PDE families, domains, BC types).

Definition

A PDE foundation model (FM) aims to be:

- Pretrained on a *diverse* collection of PDE datasets (different PDE families, domains, BC types).
- Flexible in input modalities: accept different PDEs initial conditions, boundary data, coefficients, and geometries.

Definition

A PDE foundation model (FM) aims to be:

- Pretrained on a *diverse* collection of PDE datasets (different PDE families, domains, BC types).
- Flexible in input modalities: accept different PDEs initial conditions, boundary data, coefficients, and geometries.
- Efficient at inference (fast surrogate for many queries).

Definition

A PDE foundation model (FM) aims to be:

- Pretrained on a *diverse* collection of PDE datasets (different PDE families, domains, BC types).
- Flexible in input modalities: accept different PDEs initial conditions, boundary data, coefficients, and geometries.
- Efficient at inference (fast surrogate for many queries).
- Scalable in parameters and data; able to *transfer* to downstream PDE tasks with zero/few-shot or finetuning.

Comparison: what to compare

Comparison: what to compare

There have been multiple PDE Foundational Models proposed in the past two years, varying in the following key aspects:

1. Spatial dimension(s) supported (1D/2D/3D)

Comparison: what to compare

There have been multiple PDE Foundational Models proposed in the past two years, varying in the following key aspects:

1. Spatial dimension(s) supported (1D/2D/3D)
2. Main architecture (Transformer / FNO / U-Net)

Comparison: what to compare

There have been multiple PDE Foundational Models proposed in the past two years, varying in the following key aspects:

1. Spatial dimension(s) supported (1D/2D/3D)
2. Main architecture (Transformer / FNO / U-Net)
3. How inputs are passed (varying dimensions of initial condition, boundary data, coefficients, geometries, PDE form)

Comparison: what to compare

There have been multiple PDE Foundational Models proposed in the past two years, varying in the following key aspects:

1. Spatial dimension(s) supported (1D/2D/3D)
2. Main architecture (Transformer / FNO / U-Net)
3. How inputs are passed (varying dimensions of initial condition, boundary data, coefficients, geometries, PDE form)
4. Overall solution approach (auto-regressive, operator learning, latent decoding, generative)

Comparison: what to compare

There have been multiple PDE Foundational Models proposed in the past two years, varying in the following key aspects:

1. Spatial dimension(s) supported (1D/2D/3D)
2. Main architecture (Transformer / FNO / U-Net)
3. How inputs are passed (varying dimensions of initial condition, boundary data, coefficients, geometries, PDE form)
4. Overall solution approach (auto-regressive, operator learning, latent decoding, generative)
5. Reported parameter counts and scaling behavior

Pre-trained FNO (Subramanian et al. 2023)

Pre-trained FNO (Subramanian et al. 2023)

First introduced results to depict feasibility of foundation models for PDEs in very limited settings.

1. Specifically focuses on time-indepdendent 2D PDEs.

Pre-trained FNO (Subramanian et al. 2023)

First introduced results to depict feasibility of foundation models for PDEs in very limited settings.

1. Specifically focuses on time-indepdendent 2D PDEs.
2. Main architecture consists of an FNO.

Pre-trained FNO (Subramanian et al. 2023)

First introduced results to depict feasibility of foundation models for PDEs in very limited settings.

1. Specifically focuses on time-indepdendent 2D PDEs.
2. Main architecture consists of an FNO.
3. Takes a universal PDE approach, combining the Poisson, Advection-Diffusion, and Helmholtz equations (all in $[0, 1]^2$ with periodic boundary conditions).

Pre-trained FNO (Subramanian et al. 2023)

First introduced results to depict feasibility of foundation models for PDEs in very limited settings.

1. Specifically focuses on time-indepdendent 2D PDEs.
2. Main architecture consists of an FNO.
3. Takes a universal PDE approach, combining the Poisson, Advection-Diffusion, and Helmholtz equations (all in $[0, 1]^2$ with periodic boundary conditions).
4. Input consists of a discretized forcing term, the diffusion coefficient tensor, the velocity field, and the wavenumber.

Pre-trained FNO (Subramanian et al. 2023)

First introduced results to depict feasibility of foundation models for PDEs in very limited settings.

1. Specifically focuses on time-indepdendent 2D PDEs.
2. Main architecture consists of an FNO.
3. Takes a universal PDE approach, combining the Poisson, Advection-Diffusion, and Helmholtz equations (all in $[0, 1]^2$ with periodic boundary conditions).
4. Input consists of a discretized forcing term, the diffusion coefficient tensor, the velocity field, and the wavenumber.
5. Thorough tests and experiments showed much better performance at low downstream task sample rates than models trained from scratch.

Pre-trained FNO (Subramanian et al. 2023)

First introduced results to depict feasibility of foundation models for PDEs in very limited settings.

1. Specifically focuses on time-indepdendent 2D PDEs.
2. Main architecture consists of an FNO.
3. Takes a universal PDE approach, combining the Poisson, Advection-Diffusion, and Helmholtz equations (all in $[0, 1]^2$ with periodic boundary conditions).
4. Input consists of a discretized forcing term, the diffusion coefficient tensor, the velocity field, and the wavenumber.
5. Thorough tests and experiments showed much better performance at low downstream task sample rates than models trained from scratch.
6. Generalization ability scales with parameters (max 256M) and data, but returns diminish in large data regimes.

POSEIDON (Herde et al. 2024)

POSEIDON (Herde et al. 2024)

POSEIDON learns an evolution operator

$$\mathcal{G}_t : \big(u_0(x),\, c(x),\, g(x)\big) \mapsto u(t,x),$$

where:

- $u_0(x)$ = initial condition,
- $c(x)$ = PDE coefficients (viscosity, forcing, rotation, etc.),
- $g(x)$ = boundary data.

Example PDE: 2D incompressible Navier–Stokes

$$\partial_t u + (u \cdot \nabla)u = -\nabla p + \nu \Delta u + f, \qquad \nabla \cdot u = 0.$$

POSEIDON (Herde et al. 2024)

POSEIDON learns an evolution operator

$$\mathcal{G}_t : \big(u_0(x),\, c(x),\, g(x)\big) \mapsto u(t,x),$$

where:

- $u_0(x)$ = initial condition,
- $c(x)$ = PDE coefficients (viscosity, forcing, rotation, etc.),
- $g(x)$ = boundary data.

Example PDE: 2D incompressible Navier–Stokes

$$\partial_t u + (u \cdot \nabla)u = -\nabla p + \nu \Delta u + f, \qquad \nabla \cdot u = 0.$$

2) Dimensions

POSEIDON operates on **2D spatial domains**:

$$x = (x_1, x_2) \in \Omega \subset \mathbb{R}^2.$$

It supports variable spatial resolutions through patch embeddings (also downsamples or upsamples to 128x128 grid).

3) Inputs & Handling Different PDE Types

The model receives a multi-channel input tensor:

$$X = [u_0,\ c_1, \ldots, c_k,\ g] \in \mathbb{R}^{d \times H \times W}.$$

POSEIDON handles diverse PDE families by:

- Concatenating all PDE-specific fields as input channels.
- Embedding inputs into a unified operator latent space.
- Using **time-conditioned layer norms** for continuous in time-evaluation:

$$\mathrm{LN}_t(z) = \gamma(t)\frac{z-\mu}{\sigma} + \beta(t),$$

enabling the semigroup property

$$\mathcal{G}_{t+s} = \mathcal{G}_s \circ \mathcal{G}_t.$$

3) Inputs & Handling Different PDE Types

The model receives a multi-channel input tensor:

$$X = [u_0,\ c_1, \ldots, c_k,\ g] \in \mathbb{R}^{d \times H \times W}.$$

POSEIDON handles diverse PDE families by:

- Concatenating all PDE-specific fields as input channels.
- Embedding inputs into a unified operator latent space.
- Using **time-conditioned layer norms** for continuous in time-evaluation:

$$\mathrm{LN}_t(z) = \gamma(t)\frac{z-\mu}{\sigma} + \beta(t),$$

enabling the semigroup property

$$\mathcal{G}_{t+s} = \mathcal{G}_s \circ \mathcal{G}_t.$$

- Allowing different sets of coefficients, BCs, and ICs across PDEs.

4) Main Architecture: Multiscale Operator Transformer (scOT)

- Multiresolution patching:

$$P : \mathbb{R}^{d \times H \times W} \to \mathbb{R}^{N_p \times D}.$$

- Swin-Transformer blocks (shifted windowed self-attention blocks that capture global dependencies but are more computationally feasible).
- Time-conditioned normalization to encode continuous-time evolution.
- Decoder reconstructs $u(t, x)$ from transformed patches.

4) Main Architecture: Multiscale Operator Transformer (scOT)

- Multiresolution patching:

$$P : \mathbb{R}^{d \times H \times W} \to \mathbb{R}^{N_p \times D}.$$

- Swin-Transformer blocks (shifted windowed self-attention blocks that capture global dependencies but are more computationally feasible).
- Time-conditioned normalization to encode continuous-time evolution.
- Decoder reconstructs $u(t, x)$ from transformed patches.

5) Fine-Tuning Strategy

- Pretrain on a multi-physics dataset across 15+ PDE families.
- Downstream fine-tuning:

$$\theta \leftarrow \theta_{\text{pretrained}} + \Delta\theta.$$

- Few-shot adaptation for new PDE operators with small $\Delta\theta$ observed.
- Performance scales with data and model size (max 629M).

MPP AViT (McCabe et al. 2024)

MPP AViT (McCabe et al. 2024)

1. Focuses on 2D spatiotemporal problems.

MPP AViT (McCabe et al. 2024)

1. Focuses on 2D spatiotemporal problems.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.

MPP AViT (McCabe et al. 2024)

1. Focuses on 2D spatiotemporal problems.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.
3. Has multiple input channels for uniformly discretized coefficients and solution at times $0 \leq t < T$. Set of coefficients is restricted to velocity, density, and pressure fields.

MPP AViT (McCabe et al. 2024)

1. Focuses on 2D spatiotemporal problems.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.
3. Has multiple input channels for uniformly discretized coefficients and solution at times $0 \leq t < T$. Set of coefficients is restricted to velocity, density, and pressure fields.
4. The effect of extra coefficients is hypothesized to be inferred from historical data for zero-shot performance.

MPP AViT (McCabe et al. 2024)

1. Focuses on 2D spatiotemporal problems.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.
3. Has multiple input channels for uniformly discretized coefficients and solution at times $0 \leq t < T$. Set of coefficients is restricted to velocity, density, and pressure fields.
4. The effect of extra coefficients is hypothesized to be inferred from historical data for zero-shot performance.
5. Embeds inputs into a unified operator latent space, and patches them into tokens via a sequence of strided convolutions and nonlinearities.

MPP AViT (McCabe et al. 2024)

1. Focuses on 2D spatiotemporal problems.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.
3. Has multiple input channels for uniformly discretized coefficients and solution at times $0 \leq t < T$. Set of coefficients is restricted to velocity, density, and pressure fields.
4. The effect of extra coefficients is hypothesized to be inferred from historical data for zero-shot performance.
5. Embeds inputs into a unified operator latent space, and patches them into tokens via a sequence of strided convolutions and nonlinearities.
6. Utilizes Axial transformer blocks as the backbone (axes being time and space). Allows study of transfer ability to 3D problems via minimal changes to architecture.

MPP AViT (McCabe et al. 2024)

1. Focuses on 2D spatiotemporal problems.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.
3. Has multiple input channels for uniformly discretized coefficients and solution at times $0 \leq t < T$. Set of coefficients is restricted to velocity, density, and pressure fields.
4. The effect of extra coefficients is hypothesized to be inferred from historical data for zero-shot performance.
5. Embeds inputs into a unified operator latent space, and patches them into tokens via a sequence of strided convolutions and nonlinearities.
6. Utilizes Axial transformer blocks as the backbone (axes being time and space). Allows study of transfer ability to 3D problems via minimal changes to architecture.
7. Boundary conditions sometimes encoded architecturally if periodic, otherwise inferred from history.

MPP AViT (McCabe et al. 2024)

1. Focuses on 2D spatiotemporal problems.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.
3. Has multiple input channels for uniformly discretized coefficients and solution at times $0 \leq t < T$. Set of coefficients is restricted to velocity, density, and pressure fields.
4. The effect of extra coefficients is hypothesized to be inferred from historical data for zero-shot performance.
5. Embeds inputs into a unified operator latent space, and patches them into tokens via a sequence of strided convolutions and nonlinearities.
6. Utilizes Axial transformer blocks as the backbone (axes being time and space). Allows study of transfer ability to 3D problems via minimal changes to architecture.
7. Boundary conditions sometimes encoded architecturally if periodic, otherwise inferred from history.
8. Similar favorable results are seen, with zero/few-shot capability, and scaling with data and parameters.

DPOT (Hao et al. 2024)

DPOT (Hao et al. 2024)

1. Can be hypothetically applied in spatio-temporal PDEs in d dimensions, but experiments limit to max $d = 3$.

DPOT (Hao et al. 2024)

1. Can be hypothetically applied in spatio-temporal PDEs in d dimensions, but experiments limit to max $d = 3$.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.

DPOT (Hao et al. 2024)

1. Can be hypothetically applied in spatio-temporal PDEs in d dimensions, but experiments limit to max $d = 3$.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.
3. Injects Gaussian noise into data to support more robust learning.

DPOT (Hao et al. 2024)

1. Can be hypothetically applied in spatio-temporal PDEs in d dimensions, but experiments limit to max $d = 3$.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.
3. Injects Gaussian noise into data to support more robust learning.
4. Has a Data padding/masking layer to uniformize inputs: fixes resolution, chooses maximum number of channels in the training data for input functions, and a mask channel to encode the domain geometry.

DPOT (Hao et al. 2024)

1. Can be hypothetically applied in spatio-temporal PDEs in d dimensions, but experiments limit to max $d = 3$.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.
3. Injects Gaussian noise into data to support more robust learning.
4. Has a Data padding/masking layer to uniformize inputs: fixes resolution, chooses maximum number of channels in the training data for input functions, and a mask channel to encode the domain geometry.
5. Utilizes a vision-transformer-like position encoding, a temporal aggregation kernel, followed by multi-headed Fourier attention layer to learn complex kernel integral transforms.

DPOT (Hao et al. 2024)

1. Can be hypothetically applied in spatio-temporal PDEs in d dimensions, but experiments limit to max $d = 3$.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.
3. Injects Gaussian noise into data to support more robust learning.
4. Has a Data padding/masking layer to uniformize inputs: fixes resolution, chooses maximum number of channels in the training data for input functions, and a mask channel to encode the domain geometry.
5. Utilizes a vision-transformer-like position encoding, a temporal aggregation kernel, followed by multi-headed Fourier attention layer to learn complex kernel integral transforms.
6. Similar favorable results are seen, with zero/few-shot capability, and scaling with data and parameters.

DPOT (Hao et al. 2024)

1. Can be hypothetically applied in spatio-temporal PDEs in d dimensions, but experiments limit to max $d = 3$.
2. Auto-regressively predicts $u(x, T)$ given the solution at times $< T$, i.e, learns an operator $\mathcal{G} : (u^{<T}) \mapsto u^{T}$.
3. Injects Gaussian noise into data to support more robust learning.
4. Has a Data padding/masking layer to uniformize inputs: fixes resolution, chooses maximum number of channels in the training data for input functions, and a mask channel to encode the domain geometry.
5. Utilizes a vision-transformer-like position encoding, a temporal aggregation kernel, followed by multi-headed Fourier attention layer to learn complex kernel integral transforms.
6. Similar favorable results are seen, with zero/few-shot capability, and scaling with data and parameters.
7. Biggest parameter count reached 1B parameters with more than 100k training trajectories.

PROSE-FD (Liu et al. 2024)

PROSE-FD (Liu et al. 2024)

1. Focuses on 2D spatio-temporal PDEs prevalent in fluid dynamics.

PROSE-FD (Liu et al. 2024)

1. Focuses on 2D spatio-temporal PDEs prevalent in fluid dynamics.
2. Uses a multi-modal approach, incorporating extra symbolic data as the PDE equation itself.

PROSE-FD (Liu et al. 2024)

1. Focuses on 2D spatio-temporal PDEs prevalent in fluid dynamics.
2. Uses a multi-modal approach, incorporating extra symbolic data as the PDE equation itself.
3. Uses the solution (fixed) grid at the first 10 time steps, coefficient functions, and symbolic equation data as input, and predicts the solution grid at the next 10 time steps.

PROSE-FD (Liu et al. 2024)

1. Focuses on 2D spatio-temporal PDEs prevalent in fluid dynamics.
2. Uses a multi-modal approach, incorporating extra symbolic data as the PDE equation itself.
3. Uses the solution (fixed) grid at the first 10 time steps, coefficient functions, and symbolic equation data as input, and predicts the solution grid at the next 10 time steps.
4. Encodes symbolic data via Polish notation, and the input functions via a ViT-type patch-based encoding.

PROSE-FD (Liu et al. 2024)

1. Focuses on 2D spatio-temporal PDEs prevalent in fluid dynamics.
2. Uses a multi-modal approach, incorporating extra symbolic data as the PDE equation itself.
3. Uses the solution (fixed) grid at the first 10 time steps, coefficient functions, and symbolic equation data as input, and predicts the solution grid at the next 10 time steps.
4. Encodes symbolic data via Polish notation, and the input functions via a ViT-type patch-based encoding.
5. Uses a self-attention separately on each data type, and then in a **feature-fusion** block where the two modalities are fused before being decoded via cross-attention.

PROSE-FD (Liu et al. 2024)

1. Focuses on 2D spatio-temporal PDEs prevalent in fluid dynamics.
2. Uses a multi-modal approach, incorporating extra symbolic data as the PDE equation itself.
3. Uses the solution (fixed) grid at the first 10 time steps, coefficient functions, and symbolic equation data as input, and predicts the solution grid at the next 10 time steps.
4. Encodes symbolic data via Polish notation, and the input functions via a ViT-type patch-based encoding.
5. Uses a self-attention separately on each data type, and then in a **feature-fusion** block where the two modalities are fused before being decoded via cross-attention.
6. Boundary conditions encoding not dealt with.

PROSE-FD (Liu et al. 2024)

1. Focuses on 2D spatio-temporal PDEs prevalent in fluid dynamics.
2. Uses a multi-modal approach, incorporating extra symbolic data as the PDE equation itself.
3. Uses the solution (fixed) grid at the first 10 time steps, coefficient functions, and symbolic equation data as input, and predicts the solution grid at the next 10 time steps.
4. Encodes symbolic data via Polish notation, and the input functions via a ViT-type patch-based encoding.
5. Uses a self-attention separately on each data type, and then in a **feature-fusion** block where the two modalities are fused before being decoded via cross-attention.
6. Boundary conditions encoding not dealt with.
7. Symbolic information significantly improved performance.

PROSE-PDE (Sun et al. 2024)

PROSE-PDE (Sun et al. 2024)

1. Focuses on 1D spatio-temporal ODEs.

PROSE-PDE (Sun et al. 2024)

1. Focuses on 1D spatio-temporal ODEs.
2. Aims to address the possible well-posedness issue with training a single architecture to learn to evolve a large set of distinct (sometimes unknown) PDEs from only the initial condition.

PROSE-PDE (Sun et al. 2024)

1. Focuses on 1D spatio-temporal ODEs.
2. Aims to address the possible well-posedness issue with training a single architecture to learn to evolve a large set of distinct (sometimes unknown) PDEs from only the initial condition.
3. Uses a multi-modal approach, incorporating extra symbolic data as a guess of the PDE equation itself to make it well-posed.

PROSE-PDE (Sun et al. 2024)

1. Focuses on 1D spatio-temporal ODEs.
2. Aims to address the possible well-posedness issue with training a single architecture to learn to evolve a large set of distinct (sometimes unknown) PDEs from only the initial condition.
3. Uses a multi-modal approach, incorporating extra symbolic data as a guess of the PDE equation itself to make it well-posed.
4. Uses only the initial $0 \leq t < T_0$ (fixed) grid and guess equation as input, and predicts the solution grid $\forall T > T_0$, and also the equation.

PROSE-PDE (Sun et al. 2024)

1. Focuses on 1D spatio-temporal ODEs.
2. Aims to address the possible well-posedness issue with training a single architecture to learn to evolve a large set of distinct (sometimes unknown) PDEs from only the initial condition.
3. Uses a multi-modal approach, incorporating extra symbolic data as a guess of the PDE equation itself to make it well-posed.
4. Uses only the initial $0 \leq t < T_0$ (fixed) grid and guess equation as input, and predicts the solution grid $\forall T > T_0$, and also the equation.
5. Encodes symbolic data via Polish notation.

PROSE-PDE (Sun et al. 2024)

1. Focuses on 1D spatio-temporal ODEs.
2. Aims to address the possible well-posedness issue with training a single architecture to learn to evolve a large set of distinct (sometimes unknown) PDEs from only the initial condition.
3. Uses a multi-modal approach, incorporating extra symbolic data as a guess of the PDE equation itself to make it well-posed.
4. Uses only the initial $0 \leq t < T_0$ (fixed) grid and guess equation as input, and predicts the solution grid $\forall T > T_0$, and also the equation.
5. Encodes symbolic data via Polish notation.
6. Uses a self-attention separately on each data type, and then in a **feature-fusion** block where the two modalities are fused before being decoded via cross-attention and greedy search respectively.

PROSE-PDE (Sun et al. 2024)

1. Focuses on 1D spatio-temporal ODEs.
2. Aims to address the possible well-posedness issue with training a single architecture to learn to evolve a large set of distinct (sometimes unknown) PDEs from only the initial condition.
3. Uses a multi-modal approach, incorporating extra symbolic data as a guess of the PDE equation itself to make it well-posed.
4. Uses only the initial $0 \leq t < T_0$ (fixed) grid and guess equation as input, and predicts the solution grid $\forall T > T_0$, and also the equation.
5. Encodes symbolic data via Polish notation.
6. Uses a self-attention separately on each data type, and then in a **feature-fusion** block where the two modalities are fused before being decoded via cross-attention and greedy search respectively.
7. Mainly periodic boundary conditions imposed, but varying types of initial conditions used (OOD in testing).

PROSE-PDE (Sun et al. 2024)

1. Focuses on 1D spatio-temporal ODEs.
2. Aims to address the possible well-posedness issue with training a single architecture to learn to evolve a large set of distinct (sometimes unknown) PDEs from only the initial condition.
3. Uses a multi-modal approach, incorporating extra symbolic data as a guess of the PDE equation itself to make it well-posed.
4. Uses only the initial $0 \leq t < T_0$ (fixed) grid and guess equation as input, and predicts the solution grid $\forall T > T_0$, and also the equation.
5. Encodes symbolic data via Polish notation.
6. Uses a self-attention separately on each data type, and then in a **feature-fusion** block where the two modalities are fused before being decoded via cross-attention and greedy search respectively.
7. Mainly periodic boundary conditions imposed, but varying types of initial conditions used (OOD in testing).
8. Trained on 512k training samples.

PROSE-PDE (Sun et al. 2024)

1. Focuses on 1D spatio-temporal ODEs.
2. Aims to address the possible well-posedness issue with training a single architecture to learn to evolve a large set of distinct (sometimes unknown) PDEs from only the initial condition.
3. Uses a multi-modal approach, incorporating extra symbolic data as a guess of the PDE equation itself to make it well-posed.
4. Uses only the initial $0 \leq t < T_0$ (fixed) grid and guess equation as input, and predicts the solution grid $\forall T > T_0$, and also the equation.
5. Encodes symbolic data via Polish notation.
6. Uses a self-attention separately on each data type, and then in a **feature-fusion** block where the two modalities are fused before being decoded via cross-attention and greedy search respectively.
7. Mainly periodic boundary conditions imposed, but varying types of initial conditions used (OOD in testing).
8. Trained on 512k training samples.
9. Symbolic information significantly improved performance.

PROSE-PDE (Sun et al. 2024)

1. Focuses on 1D spatio-temporal ODEs.
2. Aims to address the possible well-posedness issue with training a single architecture to learn to evolve a large set of distinct (sometimes unknown) PDEs from only the initial condition.
3. Uses a multi-modal approach, incorporating extra symbolic data as a guess of the PDE equation itself to make it well-posed.
4. Uses only the initial $0 \leq t < T_0$ (fixed) grid and guess equation as input, and predicts the solution grid $\forall T > T_0$, and also the equation.
5. Encodes symbolic data via Polish notation.
6. Uses a self-attention separately on each data type, and then in a **feature-fusion** block where the two modalities are fused before being decoded via cross-attention and greedy search respectively.
7. Mainly periodic boundary conditions imposed, but varying types of initial conditions used (OOD in testing).
8. Trained on 512k training samples.
9. Symbolic information significantly improved performance.
10. Learned inherent physics to capture unseen shocks and rarefactions.

General takeaways

General takeaways

- Any PDE-specific inputs (like coefficients, geometries, boundary/initial conditions), if incorporated explicitly, are incorporated via separate input channels (could instead have symbolic inputs from the equation).

General takeaways

- Any PDE-specific inputs (like coefficients, geometries, boundary/initial conditions), if incorporated explicitly, are incorporated via separate input channels (could instead have symbolic inputs from the equation).
- Incorporating universally relevant coefficient functions like sources, velocity, viscosity, pressure, and density in pretraining generalizes well to unseen physics with unseen coefficient functions (implies that it is possible to learn fundamental physical laws).

General takeaways

- Any PDE-specific inputs (like coefficients, geometries, boundary/initial conditions), if incorporated explicitly, are incorporated via separate input channels (could instead have symbolic inputs from the equation).
- Incorporating universally relevant coefficient functions like sources, velocity, viscosity, pressure, and density in pretraining generalizes well to unseen physics with unseen coefficient functions (implies that it is possible to learn fundamental physical laws).
- Fine-tuning can range from only prompting with downstream examples and no changes to architecture, to minimal changes like adding extra channels for new coefficient functions. Some approaches (MPP AViT) can be fine-tuned to higher dimensional problems easily.

General takeaways

- Any PDE-specific inputs (like coefficients, geometries, boundary/initial conditions), if incorporated explicitly, are incorporated via separate input channels (could instead have symbolic inputs from the equation).
- Incorporating universally relevant coefficient functions like sources, velocity, viscosity, pressure, and density in pretraining generalizes well to unseen physics with unseen coefficient functions (implies that it is possible to learn fundamental physical laws).
- Fine-tuning can range from only prompting with downstream examples and no changes to architecture, to minimal changes like adding extra channels for new coefficient functions. Some approaches (MPP AViT) can be fine-tuned to higher dimensional problems easily.
- Brute-force transformer architecture usually prevails due to large parameter settings (although there is a recent lightweight U-Net based FM with comparable performance [2]).

General takeaways

- Any PDE-specific inputs (like coefficients, geometries, boundary/initial conditions), if incorporated explicitly, are incorporated via separate input channels (could instead have symbolic inputs from the equation).
- Incorporating universally relevant coefficient functions like sources, velocity, viscosity, pressure, and density in pretraining generalizes well to unseen physics with unseen coefficient functions (implies that it is possible to learn fundamental physical laws).
- Fine-tuning can range from only prompting with downstream examples and no changes to architecture, to minimal changes like adding extra channels for new coefficient functions. Some approaches (MPP AViT) can be fine-tuned to higher dimensional problems easily.
- Brute-force transformer architecture usually prevails due to large parameter settings (although there is a recent lightweight U-Net based FM with comparable performance [2]).
- Computation usually done in unified latent space with some form of normalization to deal with variety of scales.

Open challenges & research directions

Open challenges & research directions

- Robust zero-shot generalization to unseen PDEs and geometries.

Open challenges & research directions

- Robust zero-shot generalization to unseen PDEs and geometries.
- Principled uncertainty quantification and certified error bounds for learned operators.

Open challenges & research directions

- Robust zero-shot generalization to unseen PDEs and geometries.
- Principled uncertainty quantification and certified error bounds for learned operators.
- Dataset curation: diverse, standardized pretraining corpora (multiple BCs, geometries, dimensionality).

Open challenges & research directions

- Robust zero-shot generalization to unseen PDEs and geometries.
- Principled uncertainty quantification and certified error bounds for learned operators.
- Dataset curation: diverse, standardized pretraining corpora (multiple BCs, geometries, dimensionality).
- Efficient scaling: parameter-efficient architectures vs brute-force transformers.

Open challenges & research directions

- Robust zero-shot generalization to unseen PDEs and geometries.
- Principled uncertainty quantification and certified error bounds for learned operators.
- Dataset curation: diverse, standardized pretraining corpora (multiple BCs, geometries, dimensionality).
- Efficient scaling: parameter-efficient architectures vs brute-force transformers.
- Better theoretical understanding of sample complexity and scaling laws for PDE FMs.

Open challenges & research directions

- Robust zero-shot generalization to unseen PDEs and geometries.
- Principled uncertainty quantification and certified error bounds for learned operators.
- Dataset curation: diverse, standardized pretraining corpora (multiple BCs, geometries, dimensionality).
- Efficient scaling: parameter-efficient architectures vs brute-force transformers.
- Better theoretical understanding of sample complexity and scaling laws for PDE FMs.
- Better quantitative theory behind how common physics can aide transfer learning.

Open challenges & research directions

- Robust zero-shot generalization to unseen PDEs and geometries.
- Principled uncertainty quantification and certified error bounds for learned operators.
- Dataset curation: diverse, standardized pretraining corpora (multiple BCs, geometries, dimensionality).
- Efficient scaling: parameter-efficient architectures vs brute-force transformers.
- Better theoretical understanding of sample complexity and scaling laws for PDE FMs.
- Better quantitative theory behind how common physics can aide transfer learning.
- Explore generative techniques [1].

References

- Chen et al., "FLOW MARCHING FOR A GENERATIVE PDE FOUNDATION MODEL", arXiv:2509.186114.
- Kovachki et al., "Neural Operator: Learning Maps Between Function Spaces", JMLR (2022).
- Hao et al., "DPOT: Auto-Regressive Denoising Operator Transformer for Large-Scale PDE Pre-Training", ICML 2024.
- Herde et al., "Poseidon: Efficient Foundation Models for PDEs", NeurIPS 2024.
- Liu et al., "PROSE-FD: A Multimodal PDE Foundation Model for Learning Multiple Operators for Forecasting Fluid Dynamics", NeurIPS 2024.
- Lu et al., "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators", NMI 2021.
- McCabe et al., "Multiple Physics Pretraining for Spatiotemporal Surrogate Models", NeurIPS 2024.

References

- Raissi et al., "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations", JCP 2019.
- Siddik et al., "SPUS: A Lightweight and Parameter-Efficient Foundation Model for PDEs", arXiv:2510.01370.
- Subramanian et al., "Towards Foundation Models for Scientific Machine Learning: Characterizing Scaling and Transfer Behavior", NeurIPS 2023.
- Sun et al., "Towards a Foundation Model for Partial Differential Equations (PROSE-PDE)", arXiv:2404.12355.
- Wang et al., "Latent Neural Operator for Solving Forward and Inverse PDE Problems", NeurIPS 2024.

Thank you.